Priority, Round Robin, Multilevel Queues

Lecture 9 — and Multiprocessor Scheduling

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-502: Operating Systems

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

October 29, 2026

Content Not Click

Welcome: Who Should Run Next When Not All Work Is Equal?

The big question

A laptop runs a video call, a compiler, and a backup at the same time. FCFS lets the backup block the call; SJF needs burst lengths nobody knows in advance. How do real schedulers decide — and stay fair?

- Last lecture: FCFS is simple but suffers the **convoy effect**; SJF is optimal for average WT but needs the next burst τ.
- Today: four ideas that real kernels combine — **Priority**, **RR**, **MLQ**, **MLFQ** — and what changes when there is more than one CPU.

Running thread for today

One five-process workload, scheduled by every policy, so every trade-off becomes a number rather than an adjective.

Roadmap: Five Questions

1. **Priority:** what if some processes simply matter more?
2. **Round Robin:** how do we give everyone a turn quickly?
3. **Multilevel queue:** what if processes come in classes?
4. **Multilevel feedback queue:** can the scheduler *learn* which processes are short?
5. **Multiprocessor:** which CPU, and which queue?

Silberschatz et al., Ch. 5 §5.3 (pp. 192–199) and §5.5 (p. 200 onward); Stallings, Ch. 9 §9.2 and Ch. 10 §10.1.

Concept: Matt Crick

The Running Workload W

Process	Arrival (ms)	Burst τ (ms)	Priority	Character
P1	0	8	3	medium batch job
P2	1	4	1	interactive, important
P3	2	9	4	long compute job
P4	3	5	2	system daemon
P5	4	2	5	short, low-importance

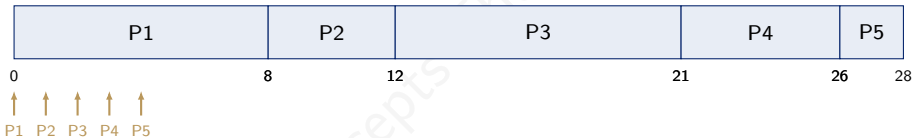
Concepts That Click

- Convention: a **lower priority number means higher priority** (Silberschatz's convention); ties go to the earlier arrival.
- Total work $= 8 + 4 + 9 + 5 + 2 = 28$ ms, so every schedule ends at 28.
- FCFS baseline (Lecture 8 method): average WT $= 11.4$ ms, average TAT $= 17.0$ ms.

Recall: The Three Metrics, Per Process

Definitions (from Lecture 8)

TAT (turnaround) = completion − arrival; **WT** (waiting) = TAT − τ; **RT** (response) = first dispatch − arrival.



- FCFS on W: WT $= 0, 7, 10, 18, 22 \Rightarrow$ average $57/5 = 11.4$ ms.
- P5 needs only 2 ms but waits 22 ms: the convoy behind P3.

SJF already ranked processes by one number — the predicted burst. Priority scheduling ranks them by any number we choose.

Priority Scheduling

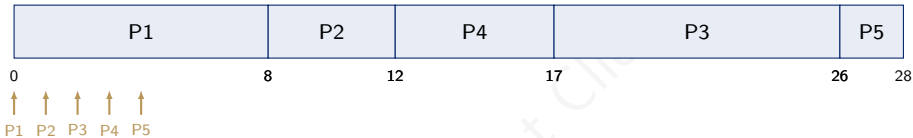
Definition

Priority scheduling associates an integer priority with each process and allocates the CPU to the ready process with the highest priority; equal priorities are broken FCFS.

- **Internal** priorities: measured quantities — time limits, memory needs, ratio of I/O to CPU burst.
- **External** priorities: set outside the OS — importance, payment, department.
- SJF is the special case where priority is the (predicted) next CPU burst: the shorter the burst, the higher the priority.
- Comes in **non-preemptive** and **preemptive** forms.

Silberschatz et al., Ch. 5 §5.3, p. 192.

Worked Example: Non-Preemptive Priority on W

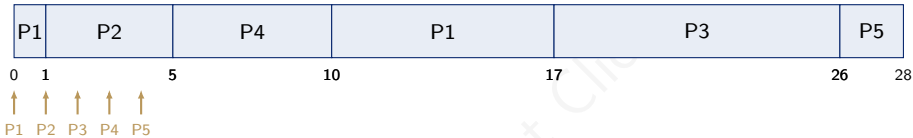


- $t = 0$: only P1 is ready; it runs its whole burst to 8.
- $t = 8$: ready $= \{$P2(1), P3(4), P4(2), P5(5)$\} \Rightarrow$ P2; at 12, P4; at 17, P3; at 26, P5.

	P1	P2	P3	P4	P5	**Average**
Completion	8	12	26	17	28	
TAT	8	11	24	14	24	16.2
WT	0	7	15	9	22	**10.6**

Contents That Cli

Worked Example: Preemptive Priority on W



- $t = 1$: P2 (priority 1) arrives and **preempts** P1 (priority 3), which keeps 7 ms of work.
- $t = 5$: P4 (2) beats P1 (3); P1 resumes at 10 and finishes at 17.

	P1	P2	P3	P4	P5	**Average**
Completion	17	5	26	10	28	
TAT	17	4	24	7	24	15.2
WT	9	0	15	2	22	**9.6**

Concepts That Click

What Priority Bought, and Who Paid

Policy on W	**avg WT**	**WT of P2**	**WT of P5**
FCFS	11.4	7	22
Non-preemptive priority	10.6	7	22
Preemptive priority	9.6	0	22

- The important process P2 now waits 0 ms.
- Non-preemptive priority could not help P2: P1 already held the CPU when P2 arrived.
- P5 still waits 22 ms — and with a steady stream of higher-priority arrivals it would wait forever.

Concepts That Cl

Starvation and Aging

Definitions

Starvation (indefinite blocking): a ready process that never gets the CPU because higher-priority work keeps arriving. **Aging**: gradually raise the priority of a process the longer it waits.

Example: bounding the wait

Priorities run from 0 (highest) to 63. Aging lowers a waiting process's priority number by 1 every 10 ms. The worst case, a process entering at 63, reaches 0 after $63 \times 10 = 630$ ms — then nothing can outrank it, so its wait is bounded.

Silberschatz et al., Ch. 5 §5.3, p. 192.

Socratic Check 1

Question

In preemptive priority on W, P1 starts first yet finishes after P2 and P4. Is P1's waiting time of 9 ms "unfair"? What would SRTF have done at $t = 1$?

- Priority judges *importance*, not length: P1 waits because two more important processes arrived.
- SRTF compares remaining times: at $t = 1$, P2's 4 ms beats P1's remaining 7 ms — the same preemption, for a different reason.
- Same mechanism, different key: that is why SJF is a priority algorithm.

Priority fixes importance but not responsiveness. Time-sharing needs a different promise: everyone gets the CPU soon, for a little while.

Round Robin (RR)

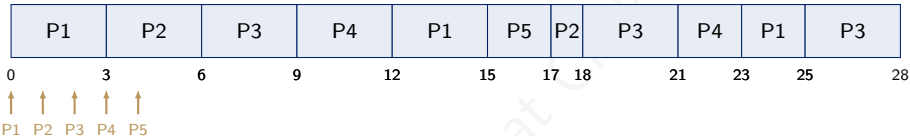
Definition

Round Robin is FCFS with preemption by a timer: each process runs for at most one **time quantum** q; if its burst is not finished, it is preempted and placed at the tail of the ready queue.

- The ready queue is a circular FIFO queue; the timer interrupt enforces the quantum (Lecture 2's timer at work).
- A burst shorter than q releases the CPU voluntarily.
- **Tie rule used today:** a process arriving at the instant a quantum expires joins the queue *before* the preempted process.

Silberschatz et al., Ch. 5 §5.3, p. 194; Stallings, Ch. 9 §9.2, p. 410.

Worked Example: RR with $q = 3$ ms on W



- $t = 3$: queue = [P2, P3, P4] (arrived during P1's slice), then P1 rejoins $\Rightarrow$ [P2, P3, P4, P1].
- $t = 6$: P5 arrived at 4, so it precedes P2 $\Rightarrow$ [P3, P4, P1, P5, P2].
- $t = 15$: queue = [P5, P2, P3, P4, P1]; P5 (2 ms) and P2 (1 ms left) finish without using a full quantum.
- $t = 21$: [P4, P1, P3] — the remainders drain: P4 to 23, P1 to 25, P3 to 28.

Concepts That Count

RR $q = 3$: The Numbers

	P1	P2	P3	P4	P5	**Average**
First dispatch	0	3	6	9	15	
Completion	25	18	28	23	17	
TAT	25	17	26	20	13	20.2
WT	17	13	17	15	11	14.6
RT	0	2	4	6	11	**4.6**

Concepts That Click

- Average RT falls from 11.4 (FCFS) to 4.6 ms.
- Average WT *rises* from 11.4 to 14.6 ms: long jobs are chopped up and wait repeatedly.
- RR trades turnaround for responsiveness — by design.

How the Quantum Changes Everything

RR on W	avg WT	avg TAT	avg RT	context switches
$q = 1$	13.0	18.6	1.2	25
$q = 2$	13.0	18.6	2.8	13
$q = 3$	14.6	20.2	4.6	10
$q = 4$	13.0	18.6	6.0	8
$q = 5$	12.6	18.2	7.4	6
$q \geq 9$	11.4	17.0	11.4	4

Concepts That Click

- Response time grows steadily with q; average WT does **not** move monotonically.
- Once q exceeds the longest burst (9 ms), RR *is* FCFS.

Context switches counted as changes of running process between consecutive slices.

The Cost of a Small Quantum

- Every preemption costs a context switch of s ms of pure overhead (save PCB, load PCB, flush caches/TLB).
- Fraction of CPU doing useful work while bursts outlast the quantum: $\dfrac{q}{q+s}$.

Example: $s = 0.5$ ms

$q = 1$: $1/1.5 = 66.7\%$ useful. $q = 3$: $3/3.5 = 85.7\%$. $q = 10$: $10/10.5 = 95.2\%$.

On W: $q = 1$'s 25 switches add 12.5 ms to a 28 ms schedule; $q = 3$'s 10 switches add 5 ms.

Rule of thumb

Make q large compared with the switch time, yet small enough that most CPU bursts (about 80%) finish within one quantum.

RR's Guarantee: Bounded Response

Property

With n processes in the ready queue and quantum q, no process waits more than $(n-1)\,q$ for its next quantum (ignoring switch time).

- Each of the other $n-1$ processes runs at most q before the queue comes back around.
- On W with $q = 3$: at most 5 processes ready, so every process is back on the CPU within $4 \times 3 = 12$ ms.
- Check: the largest RT is P5's 11 ms ≤ 12.

This bound — not the average WT — is why RR is the basis of time-sharing schedulers.

Concepts That

Socratic Check 2

Question

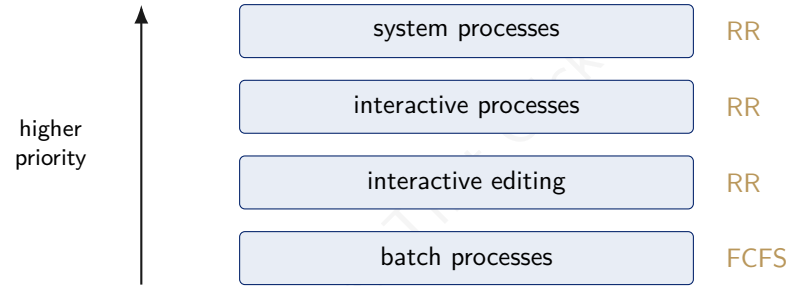
On W, $q = 2$ and $q = 4$ both give average WT 13.0 ms, yet $q = 3$ gives 14.6 ms. Does a bigger quantum always reduce waiting time?

- No. WT depends on how the quantum boundaries line up with the bursts (8, 4, 9, 5, 2).
- With $q = 4$, P2's 4 ms burst finishes in one slice; with $q = 3$ it needs two slices and waits a whole round.
- Only the limit is certain: when $q \geq$ the longest burst, RR reduces to FCFS.

One policy for every process is a compromise. Real systems split the ready queue by class — and give each class its own policy.

Concepts Click

Multilevel Queue (MLQ)



- Each process is **permanently** assigned to one queue by type (memory size, priority, process type).
- Each queue has its own algorithm; a policy *between* queues is still needed.

After Silberschatz et al., Ch. 5 §5.3, p. 196 (Fig. 5.6).

Scheduling Among the Queues

Fixed priority (preemptive)

A lower queue runs only when every higher queue is empty; a new arrival in a higher queue preempts a lower-queue process. Lower queues can starve.

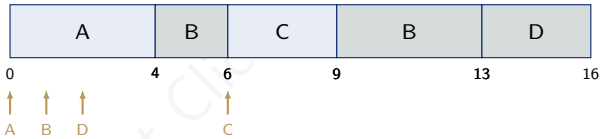
Time slicing between queues

Each queue gets a fixed share of CPU time, e.g. 80% to the foreground (RR) queue and 20% to the background (FCFS) queue. No queue starves.

- Low overhead: the class decides the queue once, at creation.
- Inflexible: a process that changes behaviour cannot change queue.

Worked Example: Two-Queue MLQ

	A	τ	queue
A	0	4	fg
B	1	6	bg
D	2	3	bg
C	6	3	fg



Foreground: RR, $q = 2$.
Background: FCFS.
Fixed priority, preemptive.

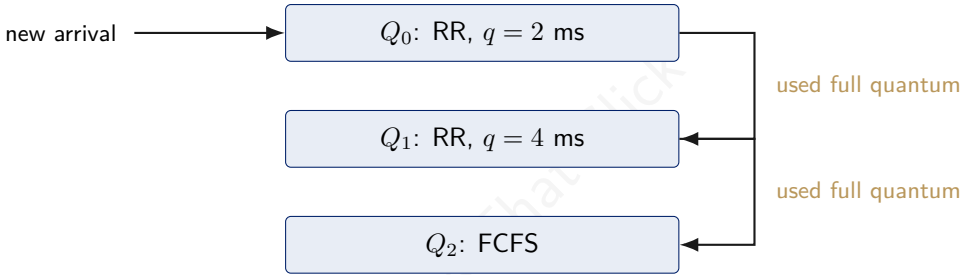
- $t = 4$: foreground empty, so background B starts.
- $t = 6$: C arrives in the foreground and **preempts** B (4 ms left); B resumes at 9.
- WT: A $= 0$, C $= 0$, B $= 12 - 6 = 6$, D $= 14 - 3 = 11$; average $17/4 = 4.25$ ms. Foreground processes never wait here.

Concepts That Cli

MLQ trusts a label fixed at creation. A feedback queue watches what each process actually does — and moves it.

Concepts Clinic

Multilevel Feedback Queue (MLFQ)



- Processes **move between queues**: a process that uses its whole quantum is demoted; I/O-bound, interactive processes stay on top.
- A process waiting too long in a low queue may be promoted (aging) — this prevents starvation.

After Silberschatz et al., Ch. 5 §5.3, p. 197; Stallings, Ch. 9 §9.2.

The Five Parameters of an MLFQ

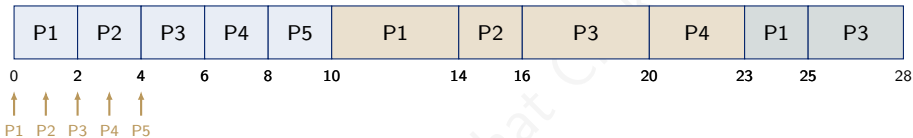
1. The **number of queues**.
2. The **scheduling algorithm** for each queue.
3. The rule for **demoting** a process to a lower queue.
4. The rule for **promoting** a process to a higher queue.
5. The rule for choosing the **entry queue** of a new process.

Why it matters

MLFQ is the most general scheme of the lecture: with suitable parameters it behaves like RR, like FCFS, or like an approximation of SJF — without being told any burst length.

Silberschatz et al., Ch. 5 §5.3, p. 197.

Worked Example: MLFQ on W



- Shading: Q_0 ($q = 2$) Q_1 ($q = 4$) Q_2 (FCFS). Higher queues preempt lower ones.
- 0–10: every process gets 2 ms in Q_0; only P5 ($\tau = 2$) finishes there. The other four are demoted.
- 10–23: Q_1 in order P1, P2, P3, P4; P2 (2 ms left) and P4 (3 ms left) finish; P1, P3 are demoted.
- 23–28: Q_2 runs P1 (2 ms) then P3 (3 ms) FCFS.

Content That Can't

MLFQ on W: The Numbers

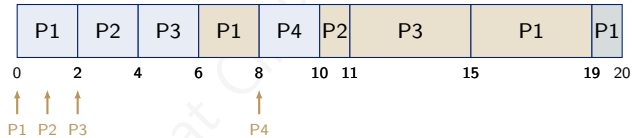
	P1	P2	P3	P4	P5	**Average**
Completion	25	16	28	23	10	
TAT	25	15	26	20	6	18.4
WT	17	11	17	15	4	12.8
RT	0	1	2	3	4	**2.0**

- The shortest job P5 finishes at 10 (FCFS: 28) — MLFQ discovered it was short without being told.
- Average RT 2.0 ms, against RR $q = 3$'s 4.6 ms, with the same number of context switches (10).

Concepts That Cut

Worked Example: A Late Short Arrival Preempts

	A	τ
P1	0	9
P2	1	3
P3	2	6
P4	8	2



Same three queues: Q_0 $q = 2$, Q_1 $q = 4$, Q_2 FCFS.

- $t = 8$: P4 enters Q_0 and preempts P1, which returns to the tail of Q_1 behind P2 and P3.
- P4 completes at 10: TAT 2, WT 0 — a new short job is served at once.
- Averages: WT $= (11 + 7 + 7 + 0)/4 = 6.25$ ms; TAT $= 45/4 = 11.25$ ms.

Concepts That Cl

Socratic Check 3

Question

A clever user notices MLFQ's demotion rule and makes a CPU-bound program issue a tiny I/O request just before each quantum expires. What happens, and how can the scheduler defend itself?

- The program never "uses a full quantum", so it is never demoted and keeps Q_0's priority — it games the scheduler.
- Defence: charge CPU time *cumulatively* at each level, and demote once the allotment is used, however it was split.
- Periodic promotion (aging) protects the processes that this program would otherwise starve.

Everything so far assumed one CPU. With several, the scheduler must also decide *where* a process runs — and moving it is not free.

Two Ways to Organize Multiple CPUs

Asymmetric multiprocessing

One master CPU runs all scheduling decisions, I/O, and kernel activity; the others execute only user code. Simple: only one CPU touches kernel data structures.

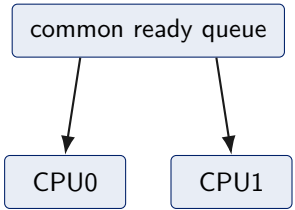
Symmetric multiprocessing (SMP)

Each CPU is **self-scheduling**: it examines the ready queue and picks its own next process. Scheduler data must be protected against simultaneous access.

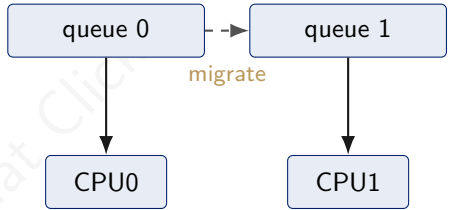
- Virtually all modern general-purpose OSes (Linux, Windows, macOS) are SMP.
- The design question becomes: one shared ready queue, or one queue per CPU?

Silberschatz et al., Ch. 5 §5.5, p. 200; Stallings, Ch. 10 §10.1, p. 453.

Common Queue vs. Per-CPU Queues



balanced, but shared lock



cache-warm, but may drift out of balance

- **Common queue:** any idle CPU takes the next process — automatic balance, but every CPU contends for one lock.
- **Per-CPU queues:** no contention and good cache reuse, but the OS must actively **load-balance**.

Processor Affinity

Definition

Processor affinity: a process prefers to keep running on the CPU it last ran on, because that CPU's cache holds its working data.

- Migrating a process invalidates its cache contents on the old CPU and repopulates them on the new one — a real cost.
- **Soft affinity:** the OS tries to keep the process on the same CPU but may move it.
- **Hard affinity:** the process restricts itself to a set of CPUs (e.g. Linux `sched_setaffinity`).
- On NUMA machines, affinity also keeps a process near the memory it uses.

Silberschatz et al., Ch. 5 §5.5, p. 200 onward.

Concepts That

Load Balancing: Push and Pull Migration

- With per-CPU queues, one CPU can sit idle while another has a long queue.
- **Push migration**: a periodic task checks loads and pushes processes from overloaded to less-busy CPUs.
- **Pull migration**: an idle CPU pulls a waiting process from a busy CPU's queue.
- Linux uses both: per-CPU runqueues with periodic *runqueue balancing*.

The tension

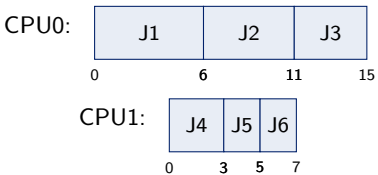
Load balancing moves processes; affinity resists moving them. A good scheduler migrates only when the imbalance outweighs the cache cost.

Silberschatz et al., Ch. 5 §5.5 (general treatment); Bovet & Cesati, Ch. 7, pp. 258–293.

Worked Example: Why Balance Matters

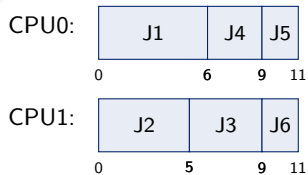
Six jobs, all ready at $t = 0$, bursts 6, 5, 4, 3, 2, 2 ms (total 22), on two CPUs.

Static split (J1–J3 on CPU0, J4–J6 on CPU1)



Finishes at 15; CPU1 idle for 8 ms.

Common queue (next job to the first free CPU)



Finishes at $11 = 22/2$: the best possible.

- The same work, the same CPUs: balance alone cuts the finish time from 15 to 11 ms.

Concepts That Click

Multicore Processors and Memory Stalls

- A **memory stall**: a core waits for data from memory after a cache miss — it can spend a large share of its time stalled.
- Multithreaded cores keep two (or more) **hardware threads**; when one stalls, the core switches to the other.
- The OS sees each hardware thread as a logical CPU; the core decides which hardware thread runs from cycle to cycle.

Two levels of scheduling

The OS chooses which software thread runs on each logical CPU; the hardware chooses which hardware thread uses the core's pipeline.

Silberschatz et al., Ch. 5 §5.5 (general treatment).

All Policies on W

Policy	avg WT	avg TAT	avg RT	P5 done at	switches
FCFS	11.4	17.0	11.4	28	4
Priority (non-preemptive)	10.6	16.2	10.6	28	4
Priority (preemptive)	9.6	15.2	7.8	28	5
RR, $q = 3$	14.6	20.2	4.6	17	10
MLFQ (2, 4, FCFS)	12.8	18.4	2.0	10	10

- No policy wins every column: the right choice depends on which criterion the system values.
- MLFQ combines RR's responsiveness with SJF-like favour for short jobs — which is why it underlies many real schedulers.

Common Misconceptions

Wrong

- "A higher priority number means more important."
- "A larger quantum always lowers waiting time."
- "RR always beats FCFS."
- "MLQ moves processes between queues."
- "More CPUs always finish proportionally sooner."

Right

- It depends on the convention; today, lower is higher.
- WT is not monotone in q (13.0, 14.6, 13.0 on W).
- RR improves RT, often at the cost of WT and TAT.
- Only MLFQ moves them; MLQ assignment is permanent.
- Only if the load is balanced (15 vs. 11 ms).

Takeaways

- **Priority** runs the most important ready process; preemption lets it react to arrivals; **aging** prevents starvation.
- **RR** bounds response time by $(n-1)q$; the quantum trades responsiveness against context-switch overhead, and at $q \geq$ the longest burst RR becomes FCFS.
- **MLQ** gives each fixed class its own queue and policy; **MLFQ** moves processes between queues by observed behaviour.
- **Multiprocessor** scheduling adds placement: SMP, common vs. per-CPU queues, **affinity**, and push/pull **load balancing**.
- Always compute: Gantt chart first, then TAT, WT, RT per process.

Comet Time Clock

Next: Concurrency and the Critical-Section Problem

- Today closes Unit I: a scheduler can stop a process at *any* timer interrupt and run another.
- Next lecture: what goes wrong when two such processes share data — race conditions — and the critical-section problem that every synchronization tool must solve.

Reading

Silberschatz et al., *Operating System Concepts*, Ch. 6 §6.1–6.2 [2]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 5 §5.1 [3].

Further Reading (Page-Cited)

- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 8e — Ch. 5 §5.3 (Priority p. 192, Round Robin p. 194, Multilevel Queue p. 196, Multilevel Feedback Queue p. 197), §5.5 (multiple-processor scheduling, p. 200) [2].
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6e — Ch. 9 §9.2 (scheduling algorithms, p. 410), Ch. 10 §10.1 (multiprocessor scheduling, p. 453) [3].
- D. P. Bovet & M. Cesati, *Understanding the Linux Kernel*, 3e — Ch. 7 (process scheduling, pp. 258–293): priorities, timeslices, per-CPU runqueues and SMP balancing [1].

Page-verified via each book's `index.md` under `master_supporting_docs/CS502/supporting_books/`.

Copyright Matt Sinclair

References I


Daniel P. Bovet and Marco Cesati.
Understanding the Linux Kernel.
O'Reilly Media, 3rd edition, 2006.


Abraham Silberschatz, Peter B. Galvin, and Greg Gagne.
Operating System Concepts.
John Wiley and Sons, 2018.


William Stallings.
Operating Systems: Internals and Design Principles.
Prentice Hall, 2001.

Concepts That Click